

How to compile a custom instruction for the RISC-V architecture using RISC-V GNU toolchain and test the result on hardware:

Modulo operator (%) is not a standard instruction of the RISC-V architecture, so it can be an extension to ALU unit.

1. Install MSYS2 on Windows 11
<http://msys2.github.io/>

This is the main console for the following commands to install/run.

2. Install the RISC-V GNU toolchain (Windows 11 OS):
<https://github.com/riscv-collab/riscv-gnu-toolchain>

```
$ git clone https://github.com/riscv/riscv-gnu-toolchain
$ git clone --recurse-submodules https://github.com/riscv/riscv-gnu-toolchain.git
```

Add `/opt/riscv/bin` to the PATH by

```
$ echo 'export PATH=/opt/riscv_custom/bin:$PATH' >> ~/.bashrc
$ source ~/.bashrc
```

```
$ cd riscv-gnu-toolchain
$ ./configure --prefix=/opt/riscv_custom
$ make -j$(nproc)
```

Check the GCC cross-compiler version

```
$ /opt/riscv custom/bin/./riscv64-unknown-elf-gcc --version
```

- ### 3. Add a custom instruction in the cross-compiler

```
$ git clone https://github.com/riscv/riscv-opcodes
```

It is assumed that a modulo instruction has to be added in RV32I which does not exist by default. The mnemonic for such an instruction would look like:

```
mod r1, r2, r3
# R[r1] = R[r2] % R[r3]
```

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0000001							rs2					rs1					000					rd					0001011				
funct7							rs2					rs1					funct3					rd					opcode				

The opcode syntax:

```
mod      rd rs1 rs2 31..25=1 14..12=0 6..2=2 1..0=3
```

0001011 is a *custom-0* opcode, which is specifically allocated for user-defined, hardware-specific instruction extensions

In `/riscv-opcodes/extensions/rv_i` modify:

```
+ mod      rd rs1 rs2 31..25=1 14..12=0 6..2=2 1..0=3
add      rd rs1 rs2 31..25=0 14..12=0 6..2=0x0C 1..0=3
```

Then, opcode file is processed to get MATCH and MASK values:

```
$ make
```

This command will generate the representation of opcodes in several formats such as SystemVerilog, Chisel and C in the `encoding.out.h` file.

Values related to the `mod` instruction are:

```
#define MATCH_MOD 0x200000b
#define MASK_MOD  0xfe00707f
0000001 xxxxx xxxxx 000 xxxxx 0001011 // Mod instruction breakdown
      1 00000 00000 000 00000 0001011 // MATCH_MOD
1111111 00000 00000 111 00000 1111111 // MASK_MOD
```

The MOD instruction is of type R, so it has the same mask as for ADD, for example. Match value can be checked with the binary representation of the instruction.

4. Modify Binutils and GDB

Open 4 files in VS editor:

- `riscv-gnu-toolchain/binutils/include/opcode/riscv-opc.h`
- `riscv-gnu-toolchain/gdb/include/opcode/riscv-opc.h`
- `riscv-gnu-toolchain/binutils/opcodes/riscv-opc.c`
- `riscv-gnu-toolchain/gdb/opcodes/riscv-opc.c`

In the header files:

```
/* Instruction opcode macros. */
+ #define MATCH_MOD 0x200000b
+ #define MASK_MOD  0xfe00707f
#define MATCH_SLLI_RV32 0x1013
// [...]
```

```

#endif /* RISC_V_ENCODING_H */
#ifdef DECLARE_INSN
+ DECLARE_INSN(mod, MATCH_MOD, MASK_MOD)
DECLARE_INSN(slli_rv32, MATCH_SLLI_RV32, MASK_SLLI_RV32)

```

In the related C files:

```

/* name, xlen, isa, operands, match, mask, match_func, pinfo. */
+ {"mod",          0, INSN_CLASS_I, "d,s,t",      MATCH_MOD, MASK_MOD, match_opcode, 0
},
{"unimp",         0, INSN_CLASS_C, "",           0, 0xffffU, match_opcode, INSN_ALIAS },

```

5. Create main.c file within /c/Users/stleraburg/riscv-gnu-toolchain to test the custom instruction:

```

#include <stdint.h>

#define LED_BASE 0x80000000
#define LED_DATA *(volatile uint32_t *) (LED_BASE)

int main(){
    int a,b,c;
    a = 5;
    b = 2;
    asm volatile (
        "mod    %[z], %[x], %[y]\n\t"
        : [z] "=r" (c)
        : [x] "r" (a), [y] "r" (b)
        );
    LED_DATA = (c==1) ? 0xFF : 0x00;
    return 0;
}

```

6. Create a linker script linker.ld in the same directory:

```

OUTPUT_ARCH(riscv)
ENTRY(_start)

MEMORY {
    ROM (rx) : ORIGIN = 0x00000000, LENGTH = 64K
    RAM (rwx) : ORIGIN = 0x40000000, LENGTH = 128K
}

```

```

_text_va = ORIGIN(ROM);
_data_va = ORIGIN(RAM);
_stack_top = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS {
    .text _text_va : {
        *(.text.start)
        *(.text)
    } > ROM

    .rodata : {
        *(.rodata)
    } > ROM

    _sdata = .;
    .data _data_va : AT(_sdata) {
        *(.data)
    } > RAM
    _edata = _sdata + SIZEOF(.data);

    _sbss = .;
    .bss (NOLOAD) : {
        *(.bss)
    } > RAM
    _ebss = .;
}

```

7. Create a small startup code in assembly language start.S

Before any standard C code can be called, the run-time environment needs to be initialized. The linker defines a number of regions in memory that the startup file uses to perform initialization.

```

.section .text.start
.global _start

_start:
    la sp, _stack_top

    la t0, _sdata
    la t1, _edata
    la t2, _data_va
    j copy_data_loop_end

```

```

copy_data_loop:
    lw t3, 0(t0)
    addi t0, t0, 4
    addi t2, t2, 4
    sw t3, -4(t2)

copy_data_loop_end:
    bltu t0, t1, copy_data_loop
    la t0, _sbss
    la t1, _ebss
    j clear_bss_loop_end

clear_bss_loop:
    sw zero, 0(t0)
    addi t0, t0, 4

clear_bss_loop_end:
    bltu t0, t1, clear_bss_loop
    call main

hang:
    j hang

```

This code

- a) Sets the stack pointer
- b) Copies .data variables from Flash memory to RAM
- c) Zeroes out the .bss variables
- d) Jumps to the main()

8. Compile and link:

```

$ /opt/riscv_custom/bin/riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 -nostartfiles
-T linker.ld -Wl,-Map=firmware.map start.S main.c -o program.elf

```

Then create a data.hex file from the program.elf file so that Vivado can understand memory initialization.

```

$ /opt/riscv_custom/bin/riscv64-unknown-elf-objcopy -O verilog program.elf data.hex

```

9. To see the instructions that are passed to the instruction memory, we use

```
riscv64-unknown-elf-objdump -d program.elf
```

This will give the following:

```
program.elf:      file format elf32-littleriscv
```

Disassembly of section .text:

00000000 <_start>:

```
0:  40020117      auipc    sp,0x40020
4:  00010113      mv      sp,sp
8:  0b800293      li      t0,184
c:  0b800313      li      t1,184
10: 40000397      auipc    t2,0x40000
14: ff038393      addi     t2,t2,-16 # 40000000
```

<_data_va>

```
18: 0140006f      j        2c <copy_data_loop_end>
```

0000001c <copy_data_loop>:

```
1c: 0002ae03      lw      t3,0(t0)
20: 00428293      addi     t0,t0,4
24: 00438393      addi     t2,t2,4
28: ffc3ae23      sw      t3,-4(t2)
```

0000002c <copy_data_loop_end>:

```
2c: fe62e8e3      bltu     t0,t1,1c <copy_data_loop>
30: 0b800293      li      t0,184
34: 0b800313      li      t1,184
38: 00c0006f      j        44 <clear_bss_loop_end>
```

0000003c <clear_bss_loop>:

```
3c: 0002a023      sw      zero,0(t0)
40: 00428293      addi     t0,t0,4
```

00000044 <clear_bss_loop_end>:

```
44: fe62ece3      bltu     t0,t1,3c <clear_bss_loop>
48: 008000ef      jal      50 <main>
```

```

0000004c <hang>:
    4c:    0000006f                j        4c <hang>

00000050 <main>:
    50:    fe010113                addi     sp,sp,-32 # 4001ffe0
<_data_va+0x1ffe0>
    54:    00112e23                sw      ra,28(sp)
    58:    00812c23                sw      s0,24(sp)
    5c:    02010413                addi     s0,sp,32
    60:    00500793                li      a5,5
    64:    fef42623                sw      a5,-20(s0)
    68:    00200793                li      a5,2
    6c:    fef42423                sw      a5,-24(s0)
    70:    fec42783                lw      a5,-20(s0)
    74:    fe842703                lw      a4,-24(s0)
    78:    02e7878b                mod     a5,a5,a4
    7c:    fef42223                sw      a5,-28(s0)
    80:    fe442703                lw      a4,-28(s0)
    84:    00100793                li      a5,1
    88:    00f71663                bne     a4,a5,94 <main+0x44>
    8c:    0ff00793                li      a5,255
    90:    0080006f                j        98 <main+0x48>
    94:    00000793                li      a5,0
    98:    80000737                lui     a4,0x80000
    9c:    00f72023                sw      a5,0(a4) # 80000000
<_stack_top+0x3ffe0000>
    a0:    00000793                li      a5,0
    a4:    00078513                mv      a0,a5
    a8:    01c12083                lw      ra,28(sp)
    ac:    01812403                lw      s0,24(sp)
    b0:    02010113                addi     sp,sp,32
    b4:    00008067                ret

```

With this information, it will be very easy to debug since we can convert these instructions to the machine binary code and compare the result of the processor.

10. Now, what we need is the .hex file containing these instructions. These have to be uploaded to the instruction memory. Upload the file to Vivado as a new design source. The implementation of the RISC-V processor is the following:

```
// program counter: holds the addr of the current instruction
module program_counter(clk, reset, PC_in, PC_out);
    input clk, reset;
    input [31:0] PC_in;
    output reg [31:0] PC_out;

    always @ (posedge clk or negedge reset)begin
        if (~reset)
            PC_out <= 32'b00;
        // else if (PC_in <= 32'b00000000000000000000000000000100)
        else
            PC_out <= PC_in;
    end
endmodule

// instruction memory
module instruction_memory(clk, reset, read_addr, instruction_out);
    input clk, reset;
    input wire [31:0] read_addr;
    output wire [31:0] instruction_out;
    //input [31:0] read_addr;
    //output reg [31:0] instruction_out;

    // (* ram_style = "block" *)
    reg [31:0] I_Mem[0:4095];

    initial begin
        // instruction_out <= 0;
        $readmemh("data.mem", I_Mem);
        // correct
    end
end
```



```

//always @ (posedge clk) begin
//    instruction_out <= I_Mem[read_addr[31:2]];
//    $display("instruction_out: ", instruction_out);
//    correct
// end

assign instruction_out = I_Mem[read_addr[31:2]];

/*
integer k;

always @ (posedge clk or negedge reset) begin
    if (~reset) begin
        for (k=0;k<64;k=k+1) begin
            I_Mem[k] <= 32'b00;
        end
    end else begin
        instruction_out <= I_Mem[read_addr];
        $display(I_Mem);
    end
end
*/

/*
always @ (posedge clk or negedge reset) begin
    if (~reset) begin
        for (k=0;k<64;k=k+1) begin
            I_Mem[k] <= 32'b00;
        end
    end

    else begin
        // R-type
        //I_Mem[0] <=
32'b0000_0000_0000_0000_0000_0000_0000_0000; // no operation

```

```

        //I_Mem[4] <= 32'b00000000_11001_10000_000_01101_0110011;
// add x13, x16, x25
        //I_Mem[8] <= 32'b0100000_00011_01000_000_00101_0110011;
// sub x5, x8, x3
        //I_Mem[12] <= 32'b00000000_00011_00010_111_00001_0110011;
// and x1, x2, x3
        //I_Mem[16] <= 32'b00000000_00101_00011_110_00100_0110011;
// or x4, x3, x5
        //I-type
        //I_Mem[20] <= 32'b0000000000011_10101_000_10110_0010011;
//addi x22, x21, 3
        //I_Mem[24] <= 32'b0000000000001_01000_110_01001_0010011;
//ori x9, x8, 1
        //L-type
        //I_Mem[28] <= 32'b0000000001111_00101_010_01000_0000011;
//lw x8, 15(x5)
        //I_Mem[32] <= 32'b0000000000011_00011_010_01001_0000011;
//lw x9, 3(x3)
        //S-type
        // I_Mem[36] <=
32'b00000000_01111_00101_010_01100_0100011; //sw x15, 12(x5)
        // I_Mem[40] <=
32'b00000000_01110_00110_010_01010_0100011; //sw x14, 10(x6)
        //SB-type
        //I_Mem[44] <= 32'h00948663; //beq x9, x9, clk
        //$display("x13 output: %d", I_Mem[4]); // address in
decimal
    end
    end
    */

endmodule

```

```

// Register file
module reg_file(clk, reset, RegWrite, Rs1, Rs2, Rd, Write_data,
read_data1, read_data2);

```

```

input clk, reset, RegWrite;
input [4:0] Rs1, Rs2, Rd;
input [31:0] Write_data;
output wire [31:0] read_data1, read_data2;

reg [31:0] Registers [31:0];

always @ (posedge clk) begin
    if (RegWrite) begin
        Registers[Rd] = Write_data;
    end
end

assign read_data1 = (Rs1 == 5'b00000) ? 32'b0 : Registers[Rs1];
assign read_data2 = (Rs2 == 5'b00000) ? 32'b0 : Registers[Rs2];

endmodule

// Immediate generator
module ImmGen (opcode, instruction, ImmExt);
    input [6:0] opcode;
    input [31:0] instruction;
    output reg [31:0] ImmExt;

    initial ImmExt <= 0;

    always @ (*) begin
        case (opcode)
            // I-type
            7'b0010011, 7'b1100111, 7'b0000011 : ImmExt <=
{{20{instruction[31]}}, instruction[31:20]};
            // S-type
            7'b0100011 : ImmExt <= {{20{instruction[31]}},
instruction[31:25], instruction[11:7]};
            // B-type

```

```

        7'b1100011 : ImmExt <= {{19{instruction[31]}}},
instruction[31], instruction[7], instruction[30:25],
instruction[11:8], 1'b0};
        // J-type
        7'b1101111 : ImmExt <= {{11{instruction[31]}}},
instruction[31], instruction[19:12], instruction[20],
instruction[30:21], 1'b0};
        // U-type
        7'b0110111, 7'b0010111 : ImmExt <= {instruction[31:12],
12'b0};
        endcase
    end

endmodule

```

```

module control_unit(instruction, RegWrite, MemRead, MemWrite, Branch,
Jump, MemtoReg, ALUSrc, Lui, PCSrc, ALUOp);
    input [6:0] instruction;
    output reg RegWrite, MemRead, MemWrite, Branch, Jump, MemtoReg,
Lui, PCSrc;
    output reg [1:0] ALUSrc;
    output reg [2:0] ALUOp;

    initial begin
        Jump <= 0;
        Lui <= 0;
        PCSrc <= 0;
        Branch <= 0;
        MemRead <= 0;
        MemtoReg <= 0;
        ALUOp <= 0;
        MemWrite <= 0;
        ALUSrc <= 0;
        RegWrite <= 0;
    end
end

```

```

always @ (*) begin
    case (instruction)
        // R-type (ADD, SUB, AND, OR, XOR, SLT, SLTU, SLL, SRL,
        SRA)
            7'b0110011 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b100000_00_00_000; //
            // I-type (ADDI, ANDI, ORI, XORI, SLTI, SLTIU, SLLI,
            SRLI, SRAI, JALR)
            7'b0010011 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b100000_01_00_001; //
            // STORE (SB, SH, SW)
            7'b0100011 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b001001_01_00_010; //
            // BRANCH (BEQ, BNE, BLT, BGE, BLTU, BGEU)
            7'b1100011 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b000100_00_00_011; // we
            compare Rs1 and Rs2 and then perform op with pc
            // LUI
            7'b0110111 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b100000_00_10_100; //
            // AUIPC
            7'b0010111 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b100000_11_00_100; //
            // JAL
            7'b1101111 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b100010_11_00_101; //
            // JALR
            7'b1100111 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b100010_11_01_101; //
            // LOAD (LW, LH, LB, LHU, LBU)
            7'b0000011 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b110001_01_00_110;
            // custom-0 (MOD)
            7'b0001011 : {RegWrite, MemRead, MemWrite, Branch, Jump,
            MemtoReg, ALUSrc, Lui, PCSrc, ALUOp} <= 13'b100000_00_00_000;
        endcase
    end
end

```

```
endmodule
```

```
module ALU_unit (A, B, Control_in, ALU_Result, zero);
    input [31:0] A, B;
    input [3:0] Control_in;
    output reg zero;
    output reg [31:0] ALU_Result;

    initial begin
        zero <= 0;
        ALU_Result <= 0;
    end

    always @ (Control_in or A or B) begin
        case (Control_in)
            4'b0000 : begin ALU_Result <= A + B; end //add
            4'b0001 : begin ALU_Result <= A - B; end //sub
            4'b0010 : begin if (A<B) ALU_Result <= 1; else ALU_Result
<= 0; end //less than
            4'b0011 : begin if ($unsigned(A) < $unsigned(B))
ALU_Result <= 1; else ALU_Result <= 0; end //less than unsigned
            4'b0100 : begin if (A>B) ALU_Result <= 1; else ALU_Result
<= 0; end // greater than
            4'b0101 : begin if ($unsigned(A) > $unsigned(B))
ALU_Result <= 1; else ALU_Result <= 0; end // greater than unsigned
            4'b0110 : begin ALU_Result <= A ^ B; end // XOR
            4'b0111 : begin ALU_Result <= A | B; end //OR
            4'b1000 : begin ALU_Result <= A & B; end //AND
            4'b1001 : begin ALU_Result <= A << B; end //SLL
            4'b1010 : begin ALU_Result <= A >> B; end //SLR
            4'b1011 : begin ALU_Result <= $signed(A) >>> B; end //SRA
            4'b1100 : begin if (A==B) ALU_Result <= 1; else
ALU_Result <= 0; end //equal
            4'b1101 : begin if (A!=B) ALU_Result <= 1; else
ALU_Result <= 0; end // not equal
            4'b1110 : begin ALU_Result <= A + B; end //pc plus 4
        endcase
    end
endmodule
```

```

        4'b1111 : begin ALU_Result <= A % B; end // mod
        // 4'b1000 : begin zero <= 0; ALU_Result <= A - (A / B) *
B; end
    endcase

    if (ALU_Result == 0) begin
        zero <= 1'b1;
    end else begin
        zero <= 1'b0;
    end

    //$display("ALU Result: %b", ALU_Result);
end

endmodule

// ALU Control
module ALU_Control (ALUOp, fun7, fun3, Control_out);
    input [2:0] ALUOp;
    input [6:0] fun7;
    input [2:0] fun3;
    output reg [3:0] Control_out;

    initial Control_out = 0;

    always @(*) begin
        casez({ALUOp, fun7, fun3})
            // ADD
            {3'b010, 7'b???????, 3'b???}, {3'b100, 7'b???????,
3'b???}, {3'b000, 7'b0000000, 3'b000}, {3'b001, 7'b???????, 3'b000},
{3'b110, 7'b???????, 3'b???} : Control_out <= 4'b0000;
            // SUB
            {3'b000, 7'b0100000, 3'b000} : Control_out <= 4'b0001;
            // less than
            {3'b011, 7'b???????, 3'b100}, {3'b001, 7'b???????,
3'b010}, {3'b000, 7'b0000000, 3'b010} : Control_out <= 4'b0010;
            // greater than

```

```

        {3'b011, 7'b???????, 3'b101} : Control_out <= 4'b0100;
        // less than unsigned
        {3'b011, 7'b???????, 3'b110}, {3'b001, 7'b???????,
3'b011},{3'b000, 7'b0000000, 3'b011} : Control_out <= 4'b0011;
        // greater than unsigned
        {3'b011, 7'b???????, 3'b111} : Control_out <= 4'b0101;
        // XOR
        {3'b000, 7'b0000000, 3'b100}, {3'b001, 7'b???????,
3'b100} : Control_out <= 4'b0110;
        // OR
        {3'b000, 7'b0000000, 3'b110}, {3'b001, 7'b???????,
3'b110} : Control_out <= 4'b0111;
        // AND
        {3'b000, 7'b0000000, 3'b111}, {3'b001, 7'b???????,
3'b111} : Control_out <= 4'b1000;
        //SLL
        {3'b000, 7'b0000000, 3'b001}, {3'b001, 7'b0000000,
3'b001} : Control_out <= 4'b1001;
        // SRL
        {3'b000, 7'b0000000, 3'b101}, {3'b001, 7'b0000000,
3'b101} : Control_out <= 4'b1010;
        // SRA
        {3'b000, 7'b0100000, 3'b101}, {3'b001, 7'b0100000,
3'b101} : Control_out <= 4'b1011;
        // BEQ (equal)
        {3'b011, 7'b???????, 3'b000} : Control_out <= 4'b1100;
        // BNE (not equal)
        {3'b011, 7'b???????, 3'b001} : Control_out <= 4'b1101;
        // JUMP (JAL, JALR) pc plus 4
        {3'b101, 7'b???????, 3'b???} : Control_out <= 4'b1110;
        // MOD
        {3'b000, 7'b0000001, 3'b000} : Control_out <= 4'b1111;
        //default: Control_out <= 4'b0000;
    endcase
    //$display("Control out: %b", Control_out);
end
endmodule

```



```

// Data memory
module data_memory (clk, reset, MemWrite, MemRead, read_addr,
Write_data, MemData_out, led_result);
    input clk, reset, MemWrite, MemRead;
    input [31:0] read_addr, Write_data;
    output [31:0] MemData_out;
    output reg [7:0] led_result;

    integer k;
    // reg [31:0] D_Memory [32767:0]; // 128KB
    reg [31:0] D_Memory [1023:0];

    always @ (posedge clk or negedge reset) begin
        if (~reset) begin
            led_result <= 7'b0;
            for (k=0; k<1024; k=k+1) begin
                D_Memory[k] <= 32'b00;
            end
        end

        else if (MemWrite) begin
            D_Memory[read_addr[9:2]] <= Write_data;
            // if (read_addr == 32'h80000000) begin
            if (read_addr[31] == 1'b1) begin
                led_result <= Write_data;
            end
        end

        end

        assign MemData_out = (MemRead) ? D_Memory[read_addr[9:2]] :
32'b00;
    endmodule

// MUXes
// sel1 = ALUSrc[0]

```

```
module mux1 (sel1, A1, B1, mux1_out);
    input sel1;
    input [31:0] A1, B1;
    output [31:0] mux1_out;
    assign mux1_out = (sel1 == 1'b0) ? A1 : B1;
endmodule
```

```
// sel2 = ALUSrc[1]
module mux2 (sel2, A2, B2, mux2_out);
    input sel2;
    input [31:0] A2, B2;
    output [31:0] mux2_out;
    assign mux2_out = (sel2 == 1'b1) ? A2 : B2;
endmodule
```

```
// sel3 = MemtoReg
module mux3 (sel3, A3, B3, mux3_out);
    input sel3;
    input [31:0] A3, B3;
    output [31:0] mux3_out;
    assign mux3_out = (sel3 == 1'b0) ? A3 : B3;
endmodule
```

```
// sel4 = PCSrc
module mux4 (sel4, A4, B4, mux4_out);
    input sel4;
    input [31:0] A4, B4;
    output [31:0] mux4_out;
    assign mux4_out = (sel4 == 1'b0) ? A4 : B4;
endmodule
```

```
// sel5 = OR out
module mux5 (sel5, A5, B5, mux5_out);
    input sel5;
    input [31:0] A5, B5;
    output [31:0] mux5_out;
    assign mux5_out = (sel5 == 1'b0) ? A5 : B5;
```

```
endmodule
```

```
// sel6 = lui
```

```
module mux6 (sel6, A6, B6, mux6_out);  
    input sel6;  
    input [31:0] A6, B6;  
    output [31:0] mux6_out;  
    assign mux6_out = (sel6 == 1'b0) ? A6 : B6;  
endmodule
```

```
// AND logic
```

```
module AND_logic (branch, zero, and_out);  
    input branch, zero;  
    output and_out;  
    assign and_out = branch & zero;  
endmodule
```

```
// OR logic
```

```
module OR_logic (jump, and_out, or_out);  
    input jump, and_out;  
    output or_out;  
    assign or_out = jump | and_out;  
endmodule
```

```
// Adder
```

```
module adder (in_1, in_2, sum_out);  
    input [31:0] in_1, in_2;  
    output [31:0] sum_out;  
    assign sum_out = in_1 + in_2;  
endmodule
```

```
module top (clk_p, clk_n, reset, result);  
    input clk_p, clk_n, reset;  
    output wire result;
```

```

    wire [31:0] PC_top, instruction_top, Rd1_top, Rd2_top,
    ImmExt_top, mux1_top, mux2_top, mux3_top, mux4_top, mux5_top,
    PCin_top, address_top, Memdata_top, WriteBack_top;
    wire [7:0] led_result;
    wire branch_top, jump_top, lui_top, PCSrc_top, MemRead_top,
    MemtoReg_top, MemWrite_top, RegWrite_top, zero_top, and_top, mux6_in;
    wire [1:0] ALUSrc_top;
    wire [2:0] ALUOp_top;
    wire [3:0] control_top;
    wire clk;

    IBUFDS clk_ibuf (.I(clk_p), .IB(clk_n), .O(clk));

    // PC
    program_counter pc(.clk(clk), .reset(reset), .PC_in(PCin_top),
    .PC_out(PC_top));

    //Instruction memory
    instruction_memory inst_memory(.clk(clk), .reset(reset),
    .read_addr(PC_top), .instruction_out(instruction_top));

    // Regfile
    reg_file reg_file(.clk(clk), .reset(reset),
    .RegWrite(RegWrite_top), .Rs1(instruction_top[19:15]),
    .Rs2(instruction_top[24:20]), .Rd(instruction_top[11:7]),
    .Write_data(WriteBack_top), .read_data1(Rd1_top),
    .read_data2(Rd2_top));

    // Immediate generator
    ImmGen imm_gen (.opcode(instruction_top[6:0]),
    .instruction(instruction_top), .ImmExt(ImmExt_top));

    // Control unit
    control_unit control_unit(.instruction(instruction_top[6:0]),
    .RegWrite(RegWrite_top), .MemRead(MemRead_top),
    .MemWrite(MemWrite_top), .Branch(branch_top), .Jump(jump_top),

```

```

.MemtoReg(MemtoReg_top), .ALUSrc(ALUSrc_top), .Lui(lui_top),
.PCSrc(PCSrc_top), .ALUOp(ALUOp_top));

    // ALU control
    ALU_Control alu_control(.ALUOp(ALUOp_top),
.fun7(instruction_top[31:25]), .fun3(instruction_top[14:12]),
.Control_out(control_top));

    // ALU
    ALU_unit alu (.A(mux2_top), .B(mux1_top),
.Control_in(control_top), .ALU_Result(address_top), .zero(zero_top));

    // ALU MUXes
    mux1 alu_mux_1(.sel1(ALUSrc_top[0]), .A1(Rd2_top),
.B1(ImmExt_top), .mux1_out(mux1_top));
    mux2 alu_mux_2(.sel2(ALUSrc_top[1]), .A2(PC_top), .B2(Rd1_top),
.mux2_out(mux2_top));

    // Adder
    adder adder(.in_1(mux4_top), .in_2(mux5_top),
.sum_out(PCin_top));

    //AND gate
    AND_logic and_gate (.branch(branch_top), .zero(zero_top),
.and_out(and_top));

    //OR gate
    OR_logic or_gate (.jump(jump_top), .and_out(and_top),
.or_out(sel5_top));

    //Memory MUX
    mux3 mem_mux(.sel3(MemtoReg_top), .A3(address_top),
.B3(Memdata_top), .mux3_out(mux3_top));

    // Adder MUXes
    mux4 adder_mux_1(.sel4(PCSrc_top), .A4(PC_top), .B4(Rd1_top),
.mux4_out(mux4_top));

```


11. Then we create the constraint file to generate clock (system clock, 200MHz) and map the reset signal to the user pushbutton and the result signal to the user LED.

```
set_property PACKAGE_PIN A17 [get_ports {result}]  
set_property IOSTANDARD LVCMOS18 [get_ports {result}]
```

```
set_property PACKAGE_PIN H9 [get_ports {clk_p}]  
set_property IOSTANDARD LVDS [get_ports {clk_p}]
```

```
set_property PACKAGE_PIN AK25 [get_ports {reset}]  
set_property IOSTANDARD LVCMOS25 [get_ports {reset}]
```

```
create_clock -period 5.000 -name sys_clk [get_ports clk_p]
```

12. After running synthesis, implementation, and generating a bitstream, the LED on the ZC706 board should light up upon pressing the corresponding pushbutton, indicating that the result of a new instruction is correct.